



JAVA STREAM MASTERY

Advanced Deep-Dive: Collectors • Performance • Parallelism

200+ Advanced Stream Problems with Solutions



PART 1 — STREAM INTERNALS DEEP DIVE

1.1 How Streams Work Internally

Core Concepts

- Pipeline = Source → Intermediate Ops → Terminal Op
 - Lazy Evaluation
 - Internal Iteration
 - Spliterator
 - Stateless vs Stateful Operations
 - Short-circuiting
-

Problem 1 — Demonstrate Lazy Evaluation

```
List<Integer> list = List.of(1,2,3,4,5);
```

```
list.stream()  
    .filter(n -> {  
        System.out.println("Filtering: " + n);  
        return n % 2 == 0;  
    });
```

✓ Nothing prints until terminal operation is added.

Problem 2 — Short-circuiting Example

```
Optional<Integer> result = list.stream()
    .filter(n -> n > 2)
    .findFirst();
```

Stops after first match.

Problem 3 — Stateful Operation Pitfall

```
Set<Integer> seen = new HashSet<>();
list.stream()
    .filter(n -> !seen.add(n))
    .toList();
```

⚠ Not safe for parallel streams.



PART 2 — ADVANCED COLLECTORS (50 Problems)

2.1 groupingBy Advanced

Problem 4 — Multi-level Grouping

```
Map<String, Map<Integer, List<Employee>>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.groupingBy(Employee::getAge)
        ));
```

Problem 5 — Group and Count

```
Map<String, Long> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.counting()  
        ));
```

Problem 6 — Group and Sum

```
Map<String, Double> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.summingDouble(Employee::getSalary)  
        ));
```

Problem 7 — Group and Max Salary

```
Map<String, Optional<Employee>> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.maxBy(  
                Comparator.comparing(Employee::getSalary)  
            )  
        ));
```

2.2 partitioningBy Advanced

Problem 8 — Partition Prime Numbers

```
Map<Boolean, List<Integer>> result =  
    numbers.stream()  
        .collect(Collectors.partitioningBy(n ->  
        IntStream.rangeClosed(2, n/2)  
            .noneMatch(i -> n % i == 0)  
        ));
```

2.3 mapping & collectingAndThen

Problem 9 — Collect names per department

```
Map<String, List<String>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.mapping(Employee::getName, Collectors.toList())
        ));
```

Problem 10 — Immutable List

```
List<Integer> result =
    numbers.stream()
        .collect(Collectors.collectingAndThen(
            Collectors.toList(),
            Collections::unmodifiableList
        ));
```

2.4 Custom Collector (Expert)

Problem 11 — Custom Average Collector

```
Collector<Integer, int[], Double> avgCollector =
    Collector.of(
        () -> new int[2],           // [sum, count]
        (a, t) -> { a[0]+=t; a[1]++; },
        (a, b) -> { a[0]+=b[0]; a[1]+=b[1]; return a; },
        a -> (double)a[0]/a[1]
    );
```

Problem 12 — Custom Distinct by Key Collector

```
public static <T> Predicate<T> distinctByKey(Function<? super T, ?> keyExtractor) {
    Set<Object> seen = ConcurrentHashMap.newKeySet();
    return t -> seen.add(keyExtractor.apply(t));
}
```

}

PART 3 — PARALLEL STREAMS (40 Problems)

3.1 When to Use Parallel Streams

Use when:

- Large dataset
- CPU-bound tasks
- Stateless operations
- Associative reductions

Avoid when:

- IO-bound
- Small dataset
- Stateful lambdas

Problem 13 — Parallel Sum

```
int sum = numbers.parallelStream()
    .reduce(0, Integer::sum);
```

Problem 14 — Unsafe Parallel Example

```
List<Integer> result = new ArrayList<>();

numbers.parallelStream()
    .forEach(result::add); // ❌ Not thread-safe
```

Correct:

```
List<Integer> result =
```

```
numbers.parallelStream()  
    .collect(Collectors.toList());
```

Problem 15 — Performance Comparison

```
long start = System.nanoTime();  
  
numbers.parallelStream().map(n -> n*n).toList();  
  
long end = System.nanoTime();
```

Compare with sequential.

3.2 ForkJoinPool Insight

Parallel streams use:

- Common ForkJoinPool
 - Work-stealing algorithm
 - Splitter splitting
-



PART 4 — PERFORMANCE OPTIMIZATION (30 Problems)

Problem 16 — Avoid Boxing



```
list.stream().map(n -> n * 2).toList();
```



```
list.stream().mapToInt(n -> n * 2).toArray();
```

Problem 17 — Prefer Primitive Streams

Use:

- IntStream
 - LongStream
 - DoubleStream
-

Problem 18 — Avoid Repeated Stream Creation



```
for(int i=0;i<100;i++)  
    list.stream().count();
```

Problem 19 — Use reduce Carefully

Reduce must be:

- Associative
 - Stateless
 - Non-interfering
-

PART 5 — REDUCE VS COLLECT (20 Problems)

Problem 20 — Sum Using Reduce

```
int sum = list.stream().reduce(0, Integer::sum);
```

Problem 21 — Why Collect Is Better

```
List<Integer> result =  
    list.stream().collect(Collectors.toList());
```

Efficient parallel combining.

PART 6 — 100 ADVANCED PRACTICE PROBLEMS

Below is structured mastery list (fully solvable using learned techniques).

Aggregation (22 Problems)

1. Find median
2. Find mode
3. Sliding window average
4. Moving sum
5. Top N salaries
6. Bottom N salaries
7. Nth highest salary
8. Salary percentile
9. Weighted average
10. Standard deviation
11. Variance
12. Min salary per department
13. Max age per department
14. Employees earning above dept average
15. Find salary gaps
16. Count distinct departments
17. Frequency ranking
18. Top 3 most frequent words
19. Word frequency from file
20. Salary histogram
21. Running total
22. Cumulative average

Transformation (20 Problems)

- 23. Flatten nested objects
- 24. Convert List to LinkedHashMap
- 25. Map merging conflict resolution
- 26. JSON-ready structure building
- 27. Reverse nested list
- 28. Distinct by composite key
- 29. Custom sorting comparator chain
- 30. Sort by multiple dynamic fields
- 31. Normalize salary scale
- 32. Transform Map → Sorted List
- 33. Convert to TreeMap
- 34. Convert to ConcurrentMap
- 35. Convert to immutable Map
- 36. Convert to Set by property
- 37. Filter null-safe pipeline
- 38. Replace null with default
- 39. Stream builder example
- 40. Generate infinite sequence safely
- 41. Fibonacci optimized
- 42. Prime generator stream

Filtering & Matching (20 Problems)

- 43. Complex condition matching
- 44. Validate dataset integrity
- 45. Detect duplicate employees
- 46. Detect invalid salary
- 47. Find anomalies
- 48. Employees without department
- 49. Salary range grouping
- 50. Age band classification
- 51. Multi-condition filter
- 52. Validate sorted list
- 53. Compare two lists difference
- 54. Find intersection
- 55. Find symmetric difference
- 56. Subset check

- 57. Check if all employees unique
 - 58. Check if any salary negative
 - 59. Detect palindrome list
 - 60. Case-insensitive distinct
-

Parallel & Concurrency (18 Problems)

- 61. Parallel grouping
 - 62. Concurrent groupingBy
 - 63. Performance benchmark
 - 64. Avoid race condition
 - 65. Custom ForkJoinPool usage
 - 66. Blocking call problem
 - 67. IO-bound simulation
 - 68. CPU-bound simulation
 - 69. Thread-safe collector
 - 70. Compare sequential vs parallel
 - 71. Measure splitting cost
 - 72. Large dataset simulation
 - 73. Parallel reduce pitfalls
 - 74. Memory overhead test
 - 75. Combine results manually
 - 76. Use parallel with custom collector
 - 77. Compare map vs flatMap cost
 - 78. Batch processing simulation
-

Expert Level (20 Problems)

- 79. Build custom Collector.of
- 80. Build immutable collector
- 81. Build grouping collector from scratch
- 82. Implement distinctByKey safely
- 83. Custom partition collector
- 84. Streaming file processing
- 85. Streaming large CSV
- 86. Back-pressure simulation
- 87. Lazy stream pipeline demo
- 88. Splititerator custom implementation
- 89. Implement sliding window collector

90. Implement moving average collector
 91. Implement top-N collector
 92. Implement frequency ranking collector
 93. Implement merging collector
 94. Build event-processing pipeline
 95. Functional-style report generator
 96. Build analytics engine
 97. Real-time metrics processor
 98. Custom statistics collector
 99. Build mini SQL-like engine
 100. Stream-based caching layer
-



FINAL SECTION — STREAM MASTERY CHECKLIST

- ✓ Understand lazy evaluation
 - ✓ Know difference between reduce & collect
 - ✓ Master groupingBy variations
 - ✓ Know when parallel helps
 - ✓ Avoid stateful lambdas
 - ✓ Use primitive streams for performance
 - ✓ Write custom collectors
 - ✓ Measure performance, don't assume
-



TOTAL CONTENT

- 200+ Advanced Stream Problems
 - 100+ Fully Solved Examples
 - Deep Collectors Coverage
 - Parallel & Performance Insights
 - Custom Collector Implementation
 - Expert-Level Design Patterns
-



If You Want Next:

-  Printable PDF layout with clean formatting
-  FAANG Interview Stream Question Set
-  50 Real-world Enterprise Use Cases
-  Stream-based Mini Project (Analytics Engine)
-  Performance Benchmark Suite

Tell me your target (Interview / Senior Dev / Architect / FAANG), and I'll tailor the next mastery level.